ASSIGNMENT 07
OBJECT ORIENTED PROGRAMMING

Name : Mahak Goyal
Roll No. : 2026010066
Course : Object oriented programming with C++
Group : 1MCA3

1. **Create a base class with a function show() that prints "Current month is October". Create a derived class that overrides the show() function to print "Next month will be November". Create objects of both classes and call their functions.**

```
#include <iostream>
using namespace std;
class Month {
public:
    void show()    {
        cout << "Current month is October" << endl;
    }
};
class NextMonth : public Month {
public:
    void show()    {
        cout << "Next month will be November" << endl;
    }
};
int main() {
    Month m;
    NextMonth n;
    m.show();
    n.show();
    return 0;
}
```

```
Current month is October
Next month will be November

--------------------------------
Process exited after 0.2474 seconds with return value 0
Press any key to continue . . .
```

2. **Create a base class Animal with a virtual function sound(). Derive classes Dog and Cat that override sound() to display their respective sounds. Call the functions using base-class pointers.**

```
#include <iostream>
using namespace std;
class Animal {
public:
    virtual void sound()    {
        cout << "Animal makes a sound" << endl;
    }
};
class Dog : public Animal {
public:
```

```
public:
    void sound()    {
        cout << "Dog barks" << endl;
    }
};
class Cat : public Animal {
public:
    void sound()    {
        cout << "Cat meows" << endl;
    }
};
int main() {
    Animal *ptr;
    Dog d;
    Cat c;
    ptr = &d;
    ptr->sound();
    ptr = &c;
    ptr->sound();
    return 0;
}
```

```
Dog barks
Cat meows

--------------------------------
Process exited after 0.2502 seconds with return value 0
Press any key to continue . . .
```

3. **Create a base class Shape with a virtual function area(). Derive classes Rectangle and Circle that override area() to calculate and display their respective areas. Demonstrate runtime polymorphism using a base-class pointer.**

```
#include <iostream>
using namespace std;
class Shape {
public:
    virtual void area()    {
        cout << "Area of shape" << endl;
    }
};
class Rectangle : public Shape {
    int length, breadth;
public:
    Rectangle(int l, int b)    {
        length = l;
        breadth = b;
    }
    void area()    {
        cout << "Area of Rectangle = "
             << length * breadth << endl;
    }
};
class Circle : public Shape {
    float radius;
```

```
public:
    Circle(float r)    {
        radius = r;
    }
    void area()    {
        cout << "Area of Circle = "
             << 3.14 * radius * radius << endl;
    }
};
int main() {
    Shape *ptr;
    Rectangle r(5, 4);
    Circle c(3);
    ptr = &r;
    ptr->area();
    ptr = &c;
    ptr->area();
    return 0;
}
```

```
Area of Rectangle = 20
Area of Circle = 28.26

--------------------------------
Process exited after 0.2392 seconds with return value 0
Press any key to continue . . .
```

4. **Create a base class Employee with data members name and basicSalary and a virtual function calculateSalary(). Derive classes PermanentEmployee and ContractEmployee that calculate salary differently. Call the overridden functions using a base-class pointer.**

```
#include <iostream>
using namespace std;
class Employee {
protected:
    string name;
    float basicSalary;
public:
    Employee(string n, float s)    {
        name = n;
        basicSalary = s;
    }
    virtual void calculateSalary()    {
        cout << "Employee Salary" << endl;
    }
};
class PermanentEmployee : public Employee {
public:
    PermanentEmployee(string n, float s)
        : Employee(n, s)    {
    }
    void calculateSalary()    {
        cout << "Permanent Employee: " << name << endl;
        cout << "Salary = " << basicSalary * 1.20 << endl;
    }
```

```
};
class ContractEmployee : public Employee {
public:
    ContractEmployee(string n, float s)
        : Employee(n, s)    {
    }
    void calculateSalary()    {
        cout << "Contract Employee: " << name << endl;
        cout << "Salary = " << basicSalary + 2000 << endl;
    }
};
int main() {
    Employee *ptr;
    PermanentEmployee p("Rahul", 30000);
    ContractEmployee c("Aman", 20000);
    ptr = &p;
    ptr->calculateSalary();
    cout << endl;
    ptr = &c;
    ptr->calculateSalary();
    return 0;
}
```

```
Car: Four wheels
Bike: Two wheels
Bus: Public transport

--------------------------------
Process exited after 0.2633 seconds with return value 0
Press any key to continue . . .
```

5. **Create a base class Vehicle with a virtual function displayInfo(). Derive classes Car, Bike, and Bus, and override the function in each class to display the vehicle's details. Demonstrate runtime polymorphism using a base-class pointer.**

```
#include <iostream>
using namespace std;
class Vehicle {
public:
    virtual void displayInfo()    {
        cout << "Vehicle details" << endl;
    }
};
class Car : public Vehicle {
public:
    void displayInfo()    {
        cout << "Car: Four wheels" << endl;
    }
};
class Bike : public Vehicle {
public:
    void displayInfo()    {
        cout << "Bike: Two wheels" << endl;
    }
};
class Bus : public Vehicle {
```

```
public:
    void displayInfo()    {
        cout << "Bus: Public transport" << endl;
    }
};
int main() {
    Vehicle *ptr;
    Car c;
    Bike b;
    Bus bus;
    ptr = &c;
    ptr->displayInfo();
    ptr = &b;
    ptr->displayInfo();
    ptr = &bus;
    ptr->displayInfo();
    return 0;
}
```

```
Car: Four wheels
Bike: Two wheels
Bus: Public transport

--------------------------------
Process exited after 0.2253 seconds with return value 0
Press any key to continue . . .
```

6. **Create a base class Person with a virtual destructor and a derived class Student with its own destructor. Create a dynamically allocated Student object using a base-class pointer and delete it. Observe and explain the order in which the destructors execute.**

```
#include <iostream>
using namespace std;
class Person {
public:
    virtual ~Person()    {
        cout << "Person destructor called" << endl;
    }
};
class Student : public Person {
public:
    ~Student()    {
        cout << "Student destructor called" << endl;
    }
};
int main() {
    Person *ptr = new Student();
    delete ptr;
    return 0;
}
```

```
Student destructor called
Person destructor called

--------------------------------
Process exited after 0.2501 seconds with return value 0
Press any key to continue . . .
```